

Engineering and Empirical Evaluation of a Dependable Concurrent Banking Ledger Engine

Formal Specification, Mutation Testing, Performance Benchmarking, and DevSecOps Automation

Ahmed Abdelmoumen Bouguessa
University of Salerno
Department of Computer Science
Salerno, Italy
a.bouguessa@studenti.unisa.it

Abstract

Financial transaction software requires strong guarantees regarding state consistency, concurrency control, and resistance to common software faults and security weaknesses. This report presents an empirical evaluation of a custom-developed concurrent Banking Ledger Engine. The evaluation combines Java Modeling Language (JML) contracts, JUnit 5 and JaCoCo for functional testing and structural coverage, PiTest for mutation analysis, the Java Microbenchmark Harness (JMH) for isolated performance measurement, and an automated GitHub Actions pipeline integrating GitGuardian, Snyk, SonarCloud, and Docker. The resulting evaluation reports 99% line coverage, a 98% mutation score, no unresolved security findings after remediation, and a measured single-threaded microbenchmark result of $0.044 \mu s$ per operation under the configured benchmark conditions. These results provide evidence of strong test adequacy and automated verification practices for the evaluated prototype, while the performance and security findings are interpreted within the scope of the implemented experiments.

CCS Concepts

• **Software and its engineering** → **Software testing and debugging**; *Formal methods*; • **Security and privacy** → *Software and application security*.

Keywords

Software Dependability, JML, Mutation Testing, DevSecOps, Continuous Integration, Spring Boot, Thread Safety

1 Introduction and Project Context

Modern financial and transactional software systems operate under strict consistency requirements, where concurrency errors, invalid state transitions, arithmetic mistakes, or insufficient input validation can lead to financial discrepancies or data corruption. Software dependability concerns the ability of a system to provide service that can justifiably be trusted, including attributes such as reliability, availability, safety, confidentiality, integrity, and maintainability [1].

Under **Option #2** of the Software Dependability course policy—which permits the dependability analysis and enhancement of a complex custom Java project—this work evaluates a concurrent **Banking Ledger Engine**. Rather than analyzing an external repository, the system was developed, tested, secured, and containerized as a custom project in order to investigate how complementary

dependability and DevSecOps techniques can be integrated into one software lifecycle.

The main objectives are:

- (1) **Design-by-Contract Specification:** Use Java Modeling Language (JML) annotations to specify method preconditions, postconditions, and class invariants for critical domain operations.
- (2) **Structural and Fault-Tolerance Testing:** Assess test adequacy using JUnit 5, JaCoCo, and bytecode mutation testing with PiTest.
- (3) **Performance Profiling:** Establish a baseline for an isolated ledger operation using the Java Microbenchmark Harness (JMH).
- (4) **Automated Security:** Integrate secret detection, dependency vulnerability scanning, and static analysis into CI/CD.
- (5) **Containerized Deployment:** Build the application as a Docker image through the automated pipeline.

The source code, test suites, workflow definitions, and project artifacts are hosted in the public repository:

<https://github.com/noumen-bouguessa/Banking-ledger-engine>.

2 Methodology and Toolchain Architecture

The evaluation combines *Software Fault Avoidance* and *Software Fault Tolerance* across distinct lifecycle phases:

- **Fault Avoidance (JML):** Declares contractual constraints on critical domain operations and documents valid state transitions.
- **Structural Testing (JUnit 5 and JaCoCo):** Evaluates functional behavior and measures source-line execution coverage.
- **Fault-Tolerance Assessment (PiTest):** Introduces synthetic code mutations to evaluate whether tests detect representative implementation faults.
- **Microbenchmarking (JMH):** Measures isolated operation latency under the configured JVM benchmark conditions.
- **Security Auditing (GitGuardian, Snyk, SonarCloud):** Detects committed secrets, checks dependencies for known vulnerabilities, and performs static analysis.
- **Automation (Docker and GitHub Actions):** Integrates testing, scanning, analysis, and containerization into an automated workflow.

This combination is important because each technique addresses a different aspect of dependability. Structural coverage measures exercised code, mutation testing evaluates whether tests detect representative implementation changes, formal contracts make important state assumptions explicit, and security tooling addresses selected classes of software and supply-chain risks.

3 Phase 1: Architectural Design and Threat Model

The Banking Ledger Engine uses a modular architecture separating account state, transaction processing, concurrency control, and REST presentation concerns.

3.1 Project Complexity and Suitability for Dependability Analysis

The project was selected as an Option #2 candidate because its behavior is not limited to simple CRUD operations. The system combines stateful financial transactions, concurrent modifications, domain invariants, REST-based input handling, automated validation, and multiple verification and deployment components.

In particular, transfers modify shared account state and must preserve balance invariants while remaining consistent under concurrent execution. This creates several dependability concerns that can be investigated independently: state validity can be expressed through JML contracts, concurrent behavior can be exercised through tests, test adequacy can be assessed through mutation analysis, synchronization costs can be studied through JMH, and application and dependency risks can be examined through security tooling.

The purpose of the project is therefore not to claim production-level banking complexity, but to provide a non-trivial custom Java system on which the dependability methods studied in the course can be applied and evaluated empirically.

3.2 Threat Model and Protection Mechanisms

The principal threats considered in the implemented prototype include concurrency anomalies, invalid account states, reflected injection through web output, and vulnerable third-party dependencies.

The implemented mitigations are:

- (1) **Concurrency Race Conditions:** Ledger modifications are protected with explicit Java ReentrantLock primitives around the relevant critical sections. This provides mutual exclusion for the protected operations; correctness ultimately depends on the completeness of the locking strategy and the scope of the critical sections.
- (2) **State Inconsistency and Negative Balances:** Domain-level validation and contractual checks constrain transaction amounts and prevent account balances from becoming negative under the specified operations.
- (3) **Reflected XSS:** The affected controller output identified by static analysis is HTML-escaped with Spring's `HtmlUtils.htmlEscape()` before being returned in the relevant response context.

- (4) **Vulnerable Dependencies:** Snyk is used in CI to detect known vulnerabilities in Maven dependency paths.

The implemented model is intentionally scoped to threats addressed by the prototype and its available verification tools. It does not constitute a complete production banking threat model; authentication, authorization, deployment hardening, audit logging, denial-of-service resistance, and distributed consistency would require additional analysis.

4 Phase 2: Formal Specification with JML

JML annotations were incorporated into the ledger domain logic to make important assumptions explicit and machine-checkable where supported by the project verification setup.

Account operations are specified with the following contractual boundaries:

- **Precondition:** Transaction amounts (Δ) must be strictly positive:

$$\Delta > 0.$$

- **Postcondition:** The resulting balance must correspond to the requested operation while preserving the non-negative balance constraint:

$$\text{Balance}_{\text{new}} = \text{Balance}_{\text{old}} \pm \Delta, \quad \text{Balance}_{\text{new}} \geq 0.$$

- **Class Invariant:** Account state maintains public invariant `balance >= 0`;

These contracts document design intent and complement dynamic unit tests by making critical domain assumptions explicit. The report distinguishes contract specification from full formal proof of the complete application: the presented JML constraints describe the properties attached to the modeled domain operations.

5 Phase 3: Automated Testing and Code Coverage

Functional behavior was evaluated with JUnit 5, while JaCoCo was used to measure source-line execution coverage.

5.1 Test Suite Execution Results

- **Total Unit Tests Executed:** 83 test cases covering edge cases, concurrent transfers, and validation exceptions.
- **Test Failures/Errors:** 0, corresponding to a 100% observed pass rate for the executed suite.
- **Line Coverage (JaCoCo):** 99% (90 of 91 reported lines).
- **SonarCloud Coverage:** 98.4% for the repository according to the reported SonarCloud result.

The remaining uncovered code is reported to be concentrated in framework-level exception handling and application bootstrap paths rather than the principal financial-domain classes. The core classes `Account`, `Ledger`, and `TransactionService` achieved complete reported instruction coverage.

Coverage is interpreted here as an execution metric rather than a correctness proof: executing a line demonstrates that the line was reached, not that every possible behavior of the line is correct.

Table 1: PiTest Mutation Testing Summary

Mutation Metric	Achieved Value
Total Mutations Generated	51
Mutations Killed by Tests	50
Mutation Score	98%
Test Strength	98%
Mutations with No Coverage	0
Total Tests Considered	83

6 Phase 4: Fault-Tolerance Assessment via Mutation Testing

To evaluate test-suite adequacy beyond structural coverage, byte-code mutation analysis was performed using PiTest with `mvnpitest:mutationCoverage`.

6.1 Mutation Analysis Metrics

PiTest generated 51 synthetic faults involving representative changes such as conditional-boundary modifications, arithmetic mutations, and return-value mutations. The test suite killed 50 mutants, resulting in a 98% mutation score.

This result indicates that the test suite is highly effective at detecting the classes of faults represented by the generated mutants. It should not be interpreted as evidence that the application is free of all possible defects, because mutation effectiveness depends on the operators generated and the behaviors covered by the tests.

7 Phase 5: Performance Microbenchmarking

Performance benchmarking was conducted using JMH under single-threaded baseline conditions with 5 warm-up iterations and 5 measurement iterations.

The configured benchmark recorded:

- **Measured Operation Time:** 0.044 μ s per operation.
- **Interpretation Scope:** The value represents the isolated operation measured by the configured JMH benchmark. It is not an end-to-end banking transaction latency and should not be interpreted as multi-threaded service throughput.

The result provides a baseline for the measured operation under the benchmark configuration. In particular, it does not account for network I/O, persistence, request parsing, serialization, database activity, or multi-thread contention.

Consequently, the measurement is useful primarily for comparing changes to the same implementation and benchmark configuration.

For full reproducibility, a future benchmark report should also record the processor model, operating system, JVM version, JMH version, number of forks, benchmark mode, and measurement duration used by the run.

8 Phase 6: DevSecOps and SAST Integration

Security checks were integrated into the CI/CD lifecycle using three complementary mechanisms:

- (1) **Secret Scanning (GitGuardian):** The repository history was scanned for accidentally committed credentials or tokens. The reported result contained no detected leaked secrets.
- (2) **Dependency Vulnerability Auditing (Snyk):** Maven dependency paths were checked against known vulnerability databases, with no vulnerable dependency paths reported by the evaluated scan.
- (3) **Static Analysis (SonarCloud):** Source code was analyzed for maintainability and security-related issues.
 - *Finding and Remediation:* SonarCloud initially reported a reflected-XSS issue (`javasecurity:S5131`) in a controller endpoint. The affected output was explicitly HTML-escaped using Spring's `HtmlUtils.htmlEscape()` in the relevant response context.
 - *Final Result:* The final reported analysis passed the configured quality gate, with an "A" rating and zero unresolved findings after remediation.

The security results demonstrate that the pipeline detected and remediated the identified static-analysis issue and that no unresolved findings were present in the final evaluated state. They should not be interpreted as proof that the application is immune to all security vulnerabilities.

9 Phase 7: CI/CD Pipeline and Containerization

Verification, security analysis, and containerization are automated through GitHub Actions using `.github/workflows/ci.yml`. The workflow is triggered by pushes and pull requests targeting the main branch.

The reported stages are:

- (1) Execute GitGuardian secret detection.
- (2) Provision Eclipse Temurin JDK 17 with Maven dependency caching.
- (3) Install the Snyk CLI and run dependency security checks.
- (4) Execute `mvncleantestjacoco:reportpitest:mutationCoverage`.
- (5) Switch to JDK 21 and execute `mvnsonar:sonar` for SonarCloud analysis.
- (6) Authenticate with Docker Hub and build/push the production image: `noumenbouguessa/banking-ledger-engine:latest`.

This automation reduces manual verification steps and makes the sequence of checks repeatable across repository changes. Threshold enforcement should be understood only where the workflow configuration explicitly fails a build when a required condition is not satisfied.

10 Discussion

The combined results illustrate why multiple dependability techniques are useful when interpreted together. JaCoCo establishes how much of the measured code was executed, whereas PiTest evaluates whether tests can detect representative implementation changes. JML makes important domain assumptions explicit, while the security tools address separate classes of operational risk. JMH contributes a performance baseline, but only for the isolated operation and environment represented by the benchmark.

The strongest empirical result is the combination of 99% reported line coverage with a 98% mutation score across the evaluated test suite. The security pipeline also demonstrates an iterative process rather than a one-time scan: a static-analysis finding was detected, remediated, and absent from the final reported state.

The evaluation nonetheless has important limitations. The performance experiment is single-threaded, the threat model is scoped to the implemented prototype, and the presented evidence does not establish correctness of every possible execution path or production-level banking security.

These limitations define clear directions for further evaluation.

11 Conclusions and Future Work

This project demonstrates a practical dependability workflow for a custom concurrent ledger prototype by combining contract specification, automated testing, mutation analysis, performance benchmarking, security scanning, and CI/CD automation.

The evaluated implementation achieved 99% reported line coverage, a 98% mutation score, and no unresolved security findings after remediation.

These results provide evidence of strong engineering and verification practices within the evaluated scope, rather than a claim of complete software correctness or production-grade banking assurance.

Future work should extend the concurrency evaluation to multiple thread counts and contention levels, introduce property-based testing for broader state-space exploration, expand the threat model to authentication and authorization as well as deployment-level threats, and investigate distributed consistency and consensus mechanisms for multi-node ledger replication.

References

- [1] University of Salerno. *Software Dependability Course Materials: Administrative Information, Threat Modeling, and Project Guidelines*. Department of Computer Science.